

1.Implement the Producer–Consumer problem using threads in Python. Demonstrate concurrent execution by creating separate producer and consumer threads and using a shared bounded buffer for communication between them.

Code:

```
import threading
import random
import time

class BoundedBuffer:
    def __init__(self, capacity):
        self.capacity = capacity
        self.slots = []
        self.cond = threading.Condition()
    def insert(self, value, who):
        with self.cond:
            while len(self.slots) == self.capacity:
                print(f" [{who}] buffer full -> waiting")
                self.cond.wait()
            self.slots.append(value)
            print(f"{who:<11} put {value:>3}  buffer = {self.slots}")
            self.cond.notify_all()
    def remove(self, who):
        with self.cond:
            while not self.slots:
                print(f" [{who}] buffer empty -> waiting")
                self.cond.wait()
            value = self.slots.pop(0)
            print(f"{who:<11} took {value:>3}  buffer = {self.slots}")
            self.cond.notify_all()
            return value

STOP = -1
shared = BoundedBuffer(capacity=4)
def make_items(name, count):
    for _ in range(count):
        shared.insert(random.randint(10, 99), name)
        time.sleep(random.uniform(0.1, 0.4))
def use_items(name, eaten):
    while True:
        value = shared.remove(name)
        if value == STOP:
            break
        eaten.append(value)
        time.sleep(random.uniform(0.2, 0.6))
if __name__ == "__main__":
```

```

tally = {"Consumer-1": [], "Consumer-2": []}
producers = [threading.Thread(target=make_items, args=(f"Producer-{n}", 5))
              for n in (1, 2)]
consumers = [threading.Thread(target=use_items, args=(name, tally[name]))
             for name in tally]
for t in producers + consumers:
    t.start()
for t in producers:
    t.join()
for _ in consumers:
    shared.insert(STOP, "Main")
for t in consumers:
    t.join()
print("\nSummary")
for name, items in tally.items():
    print(f"{name} consumed {len(items)} items: {items}")
print("All threads finished.")

```

Output:

```

kaushu@KAUSHU /c/users/dhanu/downloads
*% & 'C:\Users\dhanu\AppData\Local\Programs\Python\Python315\python.exe' 'c:\Users\dhanu\.vscode\exte
anu\Downloads\import threading.py'
Producer-1 put 91 buffer = [91]
Producer-2 put 39 buffer = [91, 39]
Consumer-1 took 91 buffer = [39]
Consumer-2 took 39 buffer = []
Producer-2 put 13 buffer = [13]
Producer-1 put 88 buffer = [13, 88]
Producer-2 put 99 buffer = [13, 88, 99]
Consumer-1 took 13 buffer = [88, 99]
Consumer-2 took 88 buffer = [99]
Producer-1 put 33 buffer = [99, 33]
Producer-2 put 89 buffer = [99, 33, 89]
Consumer-1 took 99 buffer = [33, 89]
Consumer-2 took 33 buffer = [89]
Producer-1 put 71 buffer = [89, 71]
Producer-2 put 47 buffer = [89, 71, 47]
Consumer-2 took 89 buffer = [71, 47]
Producer-1 put 94 buffer = [71, 47, 94]
Consumer-1 took 71 buffer = [47, 94]
Main put -1 buffer = [47, 94, -1]
Main put -1 buffer = [47, 94, -1, -1]
Consumer-2 took 47 buffer = [94, -1, -1]
Consumer-1 took 94 buffer = [-1, -1]
Consumer-2 took -1 buffer = [-1]
Consumer-1 took -1 buffer = []

Summary
Consumer-1 consumed 5 items: [91, 13, 99, 71, 94]
Consumer-2 consumed 5 items: [39, 88, 33, 89, 47]
All threads finished.

```